

XBoing Screen State Machine: A Z Specification

Formal Model of the Flat Mode Machine and its Shared Resources

July 2026

Contents

1	Introduction	2
2	Free Types	2
3	Screen Sets and the Attract Cycle	2
4	State	3
5	Safety Invariants (the intended contract)	3
6	Operations (as coded)	4
6.1	Attract auto-advance and the C key	4
6.2	Starting a game — level load succeeds	4
6.3	Starting a game — level load fails silently	4
6.4	Space on the title	5
6.5	Space on the high-score screen	5
6.6	Game over	5
6.7	The per-tick rule check	6
7	Conjectures: where the invariants break	6
8	Test-coverage gap	7

1 Introduction

The XBoing screen machine is *flat*. `struct sdl2_state` (`src/sdl2_state.c`) holds a single `current` mode (`SDL2ST_*`), plus `previous` and a `saved` slot for dialogue push/pop. There is no enclosing “attract” super-state: `Intro`, `Instruct`, `Demo`, `Preview`, `Keys`, `KeysEdit` and `Highscore` are all peer top-level modes. The “attract loop” is a designated *subset* with a constrained transition order (`attract_cycle[]`), i.e. a constraint on a flat machine, not a nesting.

A second, orthogonal nesting axis does exist but is out of scope here: while `current` sits on an attract mode, that mode’s driving *system* runs its own animation phase (`intro_state_t`, `demo_state_t`, ...) held inside `ctx->intro` / `ctx->demo` / `ctx->keys`, not inside `sdl2_state`. Three systems are each shared by two modes (`intro_system` drives `Intro/Instruct`; `demo_system` drives `Demo/Preview`; `keys_system` drives `Keys/KeysEdit`).

Three resources are *shared* between the attract screens and real gameplay: the `game_active` flag, the required-block grid (`block_system_still_active`), and the high-score tables (`hs_global` / `hs_personal`, chosen by `highscore_request_type`). The observed defects all arise because a transition reads a shared resource that a prior transition left in the wrong state. This model names the safety invariants the handlers must preserve and shows which operations break them.

2 Free Types

The outer modes. Legacy `BallWait/Wait` (defined but never assigned) are omitted.

$$\begin{aligned} \text{Mode} ::= & mNone \mid mPresents \mid mIntro \mid mInstruct \\ & \mid mDemo \mid mPreview \mid mKeys \mid mKeysedit \\ & \mid mHighscore \mid mBonus \mid mGame \mid mPause \\ & \mid mDialogue \mid mEdit \end{aligned}$$

The shared inner systems (descriptive: the nesting axis noted above).

$$\text{System} ::= sysNone \mid sysIntro \mid sysDemo \mid sysKeys$$

Which high-score table the display is bound to (`highscore_request_type`).

$$\text{Board} ::= bGlobal \mid bPersonal$$

fuzz has no built-in boolean; `ZBOOL` avoids the `ProB/B` keyword clash with `BOOL`.

$$\text{ZBOOL} ::= ztrue \mid zfalse$$

3 Screen Sets and the Attract Cycle

The attract screens form the auto-cycle; `Presents` is not part of it.

$$\begin{array}{|l} \hline attractModes : \mathbb{P} \text{ Mode} \\ \hline attractModes = \{mIntro, mInstruct, mDemo, mPreview, \\ \quad mKeys, mKeysedit, mHighscore\} \end{array}$$

Attract screens on which `Space` starts a game directly. `Intro` (goes to `Instruct`) and `Highscore` (conditional) are handled separately (`game_input.c:399--423`).

$$\begin{array}{|l} \hline spaceStartModes : \mathbb{P} \text{ Mode} \\ \hline spaceStartModes = \{mInstruct, mDemo, mPreview, mKeys, mKeysedit\} \end{array}$$

The cycle order used by the `C` key and each screen’s finish-timer auto-advance (`attract_cycle[]`, `game_callbacks.c:54--68`).

$attractNext : Mode \leftrightarrow Mode$
$attractNext = \{ mIntro \mapsto mInstruct, \\ mInstruct \mapsto mDemo, \\ mDemo \mapsto mKeys, \\ mKeys \mapsto mKeysedit, \\ mKeysedit \mapsto mPreview, \\ mPreview \mapsto mHighscore, \\ mHighscore \mapsto mIntro \}$

The shared system driving each attract screen (two-to-one sharing).

$driver : Mode \leftrightarrow System$
$driver = \{ mIntro \mapsto sysIntro, \\ mInstruct \mapsto sysIntro, \\ mDemo \mapsto sysDemo, \\ mPreview \mapsto sysDemo, \\ mKeys \mapsto sysKeys, \\ mKeysedit \mapsto sysKeys \}$

4 State

The flattened screen state. `blocksLoaded` abstracts `block_system_still_active` (required blocks present). `globalHasData` / `personalHasData` record whether each table held rows after the startup read (`game_init.c:327--335`); on an unprivileged install (no `/var/games`) `globalHasData` is `zfalse` while `personalHasData` may be `ztrue`. `boardChoice` is `highscore_request_type`.

$Screen$
$mode : Mode$ $gameActive : ZBOOL$ $blocksLoaded : ZBOOL$ $boardChoice : Board$ $globalHasData : ZBOOL$ $personalHasData : ZBOOL$

The machine boots into `Presents`: `ctx` is `calloc'd` so `gameActive` is `zfalse` (`game_init.c:231`); no level is loaded; both boards are read from disk (`game_init.c:327--335`) so the data flags reflect what was on disk — left unconstrained here because they are platform-dependent; and `boardChoice` defaults to `bGlobal` (`game_init.c:340`).

$Init$
$Screen'$
$mode' = mPresents$ $gameActive' = zfalse$ $blocksLoaded' = zfalse$ $boardChoice' = bGlobal$

5 Safety Invariants (the intended contract)

Stated as observer schemas so a model checker can flag any reachable state that breaks them; deliberately not folded into `Screen`, because the coded operations can reach violating states.

A game may only run once its level's required blocks are on the grid; otherwise the per-tick rule check reads an empty grid and declares the level complete.

$SafeGame$
$Screen$
$mode = mGame \Rightarrow blocksLoaded = ztrue$

The high-score screen must not display an *empty* board when a populated one is available: if it is bound to an empty global board, the personal board must also be empty (nothing better to show).

<i>SafeHighscore</i>	_____
<i>Screen</i>	_____
	$ \begin{aligned} &(mode = mHighscore \\ &\quad \wedge boardChoice = bGlobal \\ &\quad \wedge globalHasData = zfalse) \\ &\Rightarrow personalHasData = zfalse \end{aligned} $

`game_active` may legitimately stay true on the game-over **Highscore** display (it gates score submission), but it must not propagate to the *other* attract screens; otherwise the **Highscore Space** routing misfires.

<i>SafeAttract</i>	_____
<i>Screen</i>	_____
	$ \begin{aligned} &(mode \in attractModes \wedge mode \neq mHighscore) \\ &\Rightarrow gameActive = zfalse \end{aligned} $

6 Operations (as coded)

6.1 Attract auto-advance and the C key

Both the finish-timer (`highscore_cb_on_finished` and `peers`) and the C key advance the cycle. Crucially, this transition *preserves* `gameActive` — the auto-advance from **Highscore** does not clear it (`game_callbacks.c:750--759`). Entering **Highscore** binds the display to `boardChoice` but performs no disk read and no global-to-personal fallback on the attract path (`game_modes.c:1139--1142`).

<i>AttractAdvance</i>	_____
$\Delta Screen$	_____
	$ \begin{aligned} &mode \in \text{dom } attractNext \\ &mode' = attractNext(mode) \\ &gameActive' = gameActive \\ &blocksLoaded' = blocksLoaded \\ &boardChoice' = boardChoice \\ &globalHasData' = globalHasData \\ &personalHasData' = personalHasData \end{aligned} $

6.2 Starting a game — level load succeeds

`start_new_game` (`game_modes.c:90--166`) sets `gameActive` and, on a successful load, populates the grid via the `on_add_block` callback. This is the correct path.

<i>StartNewGameOk</i>	_____
$\Delta Screen$	_____
	$ \begin{aligned} &mode \in spaceStartModes \\ &mode' = mGame \\ &gameActive' = ztrue \\ &blocksLoaded' = ztrue \\ &boardChoice' = boardChoice \\ &globalHasData' = globalHasData \\ &personalHasData' = personalHasData \end{aligned} $

6.3 Starting a game — level load fails silently

If the level file cannot be resolved or parsed (`game_modes.c:128--136`) `start_new_game` only warns to `stderr` and still enters **Game** with the grid left cleared.

<i>StartNewGameFail</i> $\Delta Screen$
<i>mode</i> $\in$ <i>spaceStartModes</i> <i>mode'</i> = <i>mGame</i> <i>gameActive'</i> = <i>ztrue</i> <i>blocksLoaded'</i> = <i>zfalse</i> <i>boardChoice'</i> = <i>boardChoice</i> <i>globalHasData'</i> = <i>globalHasData</i> <i>personalHasData'</i> = <i>personalHasData</i>

6.4 Space on the title

<i>IntroSpace</i> $\Delta Screen$
<i>mode</i> = <i>mIntro</i> <i>mode'</i> = <i>mInstruct</i> <i>gameActive'</i> = <i>gameActive</i> <i>blocksLoaded'</i> = <i>blocksLoaded</i> <i>boardChoice'</i> = <i>boardChoice</i> <i>globalHasData'</i> = <i>globalHasData</i> <i>personalHasData'</i> = <i>personalHasData</i>

6.5 Space on the high-score screen

The `Space` handler routes on `gameActive` and clears it here (`game_input.c:411--423`): a live game-over returns to `Intro`; otherwise it starts a game.

<i>HighscoreSpaceReturn</i> $\Delta Screen$
<i>mode</i> = <i>mHighscore</i> <i>gameActive</i> = <i>ztrue</i> <i>mode'</i> = <i>mIntro</i> <i>gameActive'</i> = <i>zfalse</i> <i>blocksLoaded'</i> = <i>blocksLoaded</i> <i>boardChoice'</i> = <i>boardChoice</i> <i>globalHasData'</i> = <i>globalHasData</i> <i>personalHasData'</i> = <i>personalHasData</i>

<i>HighscoreSpaceStart</i> $\Delta Screen$
<i>mode</i> = <i>mHighscore</i> <i>gameActive</i> = <i>zfalse</i> <i>mode'</i> = <i>mGame</i> <i>gameActive'</i> = <i>ztrue</i> <i>blocksLoaded'</i> = <i>ztrue</i> <i>boardChoice'</i> = <i>boardChoice</i> <i>globalHasData'</i> = <i>globalHasData</i> <i>personalHasData'</i> = <i>personalHasData</i>

6.6 Game over

`game_rules_ball_died` (`game_rules.c:294--296`) leaves `gameActive` true and shows `Highscore`.

GameOver

Δ Screen

```
mode = mGame
mode' = mHighscore
gameActive' = ztrue
blocksLoaded' = blocksLoaded
boardChoice' = boardChoice
globalHasData' = globalHasData
personalHasData' = personalHasData
```

6.7 The per-tick rule check

Every Game tick, `game_rules_check` (`game_rules.c:330--340`) sends the machine to Bonus the instant no required blocks remain. It is *unconditional* — no guard that a level was actually loaded.

GameRulesToBonus

Δ Screen

```
mode = mGame
blocksLoaded = zfalse
mode' = mBonus
gameActive' = gameActive
blocksLoaded' = blocksLoaded
boardChoice' = boardChoice
globalHasData' = globalHasData
personalHasData' = personalHasData
```

7 Conjectures: where the invariants break

Each defect is a reachable state that violates one safety schema. A model checker (probcli) explores Init plus the operations and reports the trace; the prose states the trace and the missing guard.

Defect 2 — empty high-score board in the attract cycle

The boards *are* read at startup, so “never loaded” is not the cause. On an unprivileged install `globalHasData = zfalse` and `personalHasData = ztrue`, and `boardChoice = bGlobal` from Init. The trace *Init ; AttractAdvance** reaches `mode = mHighscore` still bound to the empty global board, violating `SafeHighscore`. **Missing logic:** on the attract path, `mode_highscore_enter` must fall back to `bPersonal` when the global board is inactive/empty — the fallback at `game_modes.c:1134` fires only post-game-over.

Defect 3 — first Space advances the loop

In a *pure* attract cycle `gameActive` is always `zfalse` (set only by `StartNewGame*`, cleared only by the Highscore Space handler), so `IntroSpace` (`Intro → Instruct`) is the *intended* PR #172 behavior, not a bug. The genuine violation requires a prior game over: the trace *... ; GameOver ; AttractAdvance** carries `gameActive = ztrue` out of Highscore into Intro and beyond (`SafeAttract` violated at Intro); the next time the cycle reaches Highscore, `HighscoreSpaceReturn` fires and the first Space goes to Intro instead of starting a game. **Missing postcondition:** the Highscore finish-timer auto-advance (`highscore_cb_on_finished`) must set `gameActive' = zfalse` when it is not a Space-driven new game.

Defect 1 — Space lands in Bonus with an empty grid

`StartNewGameOk` establishes `blocksLoaded`, so the model exonerates the *happy-path* Space-to-game edge. The violation is `StartNewGameFail`: a silent level-load failure (`game_modes.c:128--136`) enters Game with `blocksLoaded = zfalse`, and the very next `GameRulesToBonus` tick drops to Bonus (`SafeGame` violated). **Missing guard:** `start_new_game` must not enter Game on a failed load, and/or `game_rules_check` must require a positive “level populated” precondition rather than trusting the grid.

8 Test-coverage gap

The mode suite (`tests/test_integration_modes.c`) force-transitions to each mode and ticks for *no-crash* only; it asserts nothing about the shared resources. None of the three invariants is guarded:

- No test drives *Init* ; *AttractAdvance** to **Highscore** with an empty global board and asserts the display falls back to a populated personal board (**SafeHighscore**).
- No test drives **GameOver** then auto-cycles and asserts **gameActive** is cleared before the next attract screen (**SafeAttract**), nor that the first **Space** after an auto-cycled game-over starts a game.
- No test asserts entering **Game** on a failed level load does not immediately fall to **Bonus** (**SafeGame**); the one bonus-fixture test deliberately uses an *empty* grid, exercising the opposite.

Each invariant should become a checked postcondition of the corresponding handler and a regression test.